Weather-Based Prediction of Wind Turbine Energy Output:

A Next-Generation Approach to Renewable Energy Management

Project Description:

The increasing demand for clean and sustainable energy has made renewable energy sources a global priority. Among various renewable sources, wind energy plays a significant role due to its efficiency, scalability, and environmental benefits. However, accurate prediction of wind turbine energy output remains a challenging task because it depends on multiple dynamic environmental factors such as wind speed, atmospheric pressure, temperature, humidity, and theoretical power curves.

This project focuses on developing a Machine Learning–based system to predict wind turbine energy output using weather and turbine operational parameters. The system is designed to assist in smart renewable energy management by providing reliable power output predictions based on real-time or simulated weather inputs.

The model is trained using the Wind Turbine SCADA dataset (T1.csv), which contains turbine operational data such as wind speed, theoretical power curve values, and active power generation. Data preprocessing techniques are applied to clean and prepare the dataset, followed by training a regression model to learn the relationship between weather conditions and energy output.

The trained model is integrated into a web-based application built using Flask. The application provides an interactive dashboard where users can:

- Select a city to view weather details
- Enter wind turbine parameters
- Predict energy output instantly

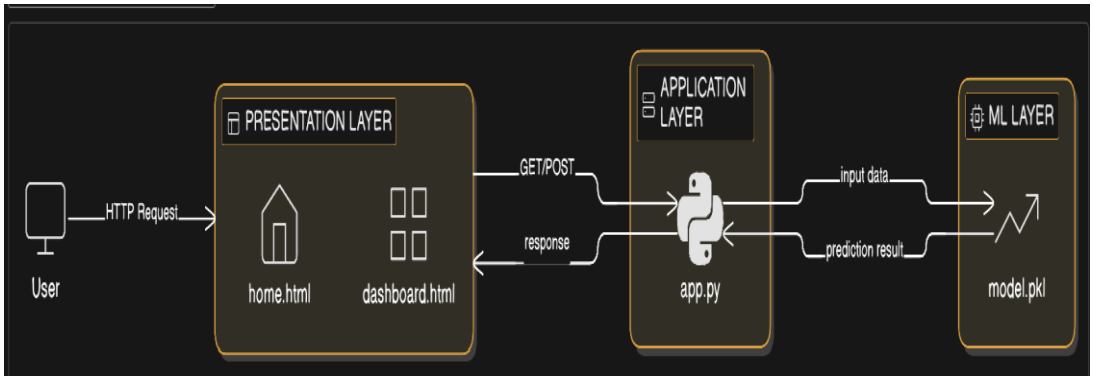
The system architecture follows a three-tier approach consisting of:

1. Presentation Layer (HTML/CSS frontend)
2. Application Layer (Flask backend)
3. Machine Learning Layer (Regression model)

By combining machine learning with web technologies, the project delivers a real-time, user-friendly wind energy prediction platform. This solution supports improved energy planning, optimized turbine performance monitoring, and better renewable resource management.

The proposed system demonstrates how predictive analytics can enhance sustainable energy operations and contribute to next-generation renewable energy management systems.

Technical Architecture:



Pre requisites:

To complete this project, you must require the following software, concepts, and packages:

- **VS Code (Visual Studio Code):**
  - Install **Visual Studio Code** for writing and running the project.
  - Install the **Python Extension** in VS Code.
  - Open VS Code Terminal as Administrator and install the following packages:

- **Python packages:**
  - Type pip install numpy and click enter.
  - Type pip install pandas and click enter.
  - Type pip install scikit-learn and click enter.
  - Type pip install flask and click enter.
  - Type pip install joblib and click enter.

Prior Knowledge:

You must have prior knowledge of following topics to complete this project.

- **ML Concepts**
  - Supervised learning: https://www.javatpoint.com/supervised-machine-learning
  - Linear Regression: https://www.javatpoint.com/machine-learning-linear-

regression

- Regression and classification: https://www.javatpoint.com/classification-vs-regression-in-machine-learning
- Data Preprocessing: https://www.javatpoint.com/data-preprocessing-machine-learning
- Model Training and Evaluation : https://www.analyticsvidhya.com/blog/2021/03/a-comprehensive-guide-to-model-evaluation-in-machine-learning/
- Scikit-Learn Basics: https://scikit-learn.org/stable/tutorial/basic/tutorial.html
- Pandas and NumPy Basics:
  Pandas: https://pandas.pydata.org/docs/getting_started/index.html
  NumPy: https://numpy.org/doc/stable/user/quickstart.html
- HTML & CSS Basics https://www.w3schools.com/html/
  https://www.w3schools.com/css/
- Flask Basics : https://flask.palletsprojects.com/en/latest/tutorial/

Project Objectives:

By the end of this project you will:

- Understand the fundamental concepts and techniques used in Machine Learning (especially Regression)
- Gain practical knowledge in handling and analyzing real-world wind turbine datasets.
- Learn data preprocessing techniques such as handling missing values and feature selection.
- Understand how to train, test, and evaluate a regression model.
- Learn how to integrate a Machine Learning model into a web application.
- Gain knowledge of building a web-based prediction system using Flask

Project Flow:

- User interacts with the Web UI to enter wind parameters (Wind Speed and Theoretical Power).
- The entered input is sent to the backend server using HTTP POST method.
- The Flask backend processes the input data.
- The trained Machine Learning model analyzes the input values.
- The prediction result (Energy Output) is generated.
- The predicted output is displayed on the User Interface.

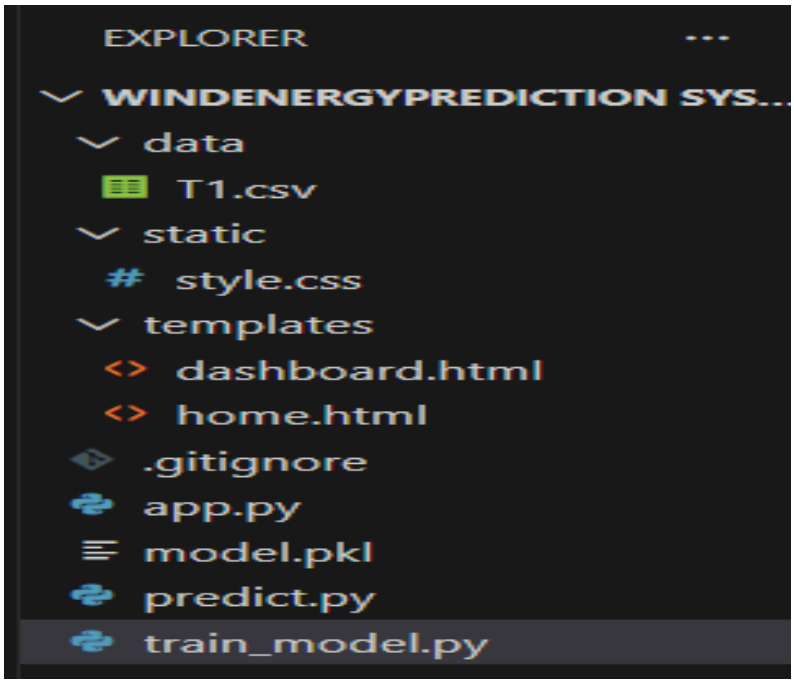
To accomplish this, we have to complete all the activities listed below:

- Data collection
  - Collect the Wind Turbine dataset (T1.csv).
  - Ensure dataset contains features such as Wind Speed, Theoretical Power, and Power Output.
- Visualizing and Analyzing data

    - Perform Univariate analysis (analyzing individual features).
    - Perform Bivariate analysis (relationship between wind speed and power output).
    - Perform Multivariate analysis (relationship between multiple input variables).
    - Perform Descriptive analysis (mean, median, standard deviation, etc).
- Data pre-processing
    - Checking for null values in the dataset.
    - Handling missing values.
    - Feature selection (selecting Wind Speed and Theoretical Power).
    - Splitting data into training and testing sets.
- Model building
    - Import the model building libraries.
    - Initializing the Linear Regression model.
    - Train the model using training data.
    - Test the model using test data.
    - Evaluate performance of the model (R^2 Score / Mean Squared Error).
    - Save the trained model as model.pkl
- Application Building
    - Create an HTML files (home.html and dashboard.html).
    - Design the frontend using HTML & CSS.
    - Build backend logic using Python and Flask.
    - Load the saved model into the Flask application.
    - Connect the model with user inputs.
    - Display prediction results on the webpage.

Project Structure:

Create the Project folder which contains files as shown below



- We are building a Flask application which requires HTML pages stored inside the

templates folder and a Python script app.py for backend scripting..

- model.pkl is our saved trained machine learning model .This model is created using Scikit-learn and is later loaded inside app.py for prediction.
- T1.csv is the Wind Turbine dataset used for training the model.
- train_model.py This file is responsible for: Loading the dataset , Preprocessing the data , Training the Linear Regression model , Saving the trained model as model.pkl .
- predict.py This file is used to: Load the saved model , Test prediction functionality independently , Verify the model performance before Flask integration.
- templates Folder Contains frontend HTML files: home.html → Landing page , dashboard.html → User input & prediction page.
- static Folder Contains static files such as: style.css → Styling the web pages.

Milestone 1: Data Collection

Machine Learning depends heavily on data. It is the most crucial aspect that makes algorithm training possible. This section allows you to collect and prepare the required dataset for building the wind energy prediction model.

.**Activity 1: Download the dataset**

There are many popular open sources for collecting the data. Eg: kaggle.com, UCI repository, etc.

In this project, we have used the Wind Turbine SCADA Dataset (T1.csv).

The dataset contains: Wind Speed , Theoretical Power Curve (KWh) , Actual Power Output.

Link: https://www.kaggle.com/search?q=wind+turbine+in%3Adatasets

Milestone 2: Model Training

In this milestone, a machine learning model is trained using sample wind turbine data. The goal is to build a regression model that can predict wind energy output based on input features such as theoretical power and wind speed.

File Used:

train_model.py

Libraries Used:

- NumPy
- LinearRegression from Scikit-learn
- Pickle

Step 1: Import Required Libraries

Explanation:

- numpy → Used to create numerical arrays.
- LinearRegression → Regression algorithm.
- `pickle` → Used to save trained model.

Purpose: To load necessary tools required for training and saving the model.

```
train_model.py > ...
import numpy as np
from sklearn.linear_model import LinearRegression
import pickle

# Sample training data
X = np.array([
    [100, 5],
    [200, 7],
    [300, 9],
    [400, 11],
    [500, 13]
])

y = np.array([120, 250, 380, 520, 680])

model = LinearRegression()
model.fit(X, y)

pickle.dump(model, open("model.pkl", "wb"))

print("Model trained and saved successfully!")
```

Step 2: Create Training Dataset

X = np.array([[100, 5],[200, 7], [300, 9],[400, 11],[500, 13]])

y = np.array([120, 250, 380, 520, 680])

Explanation:

- X contains input features : Theoretical Power and Wind Speed.

Purpose: To provide data so the model can learn the relationship between input and output.

Step 3: Initialize the Model

model = LinearRegression()

Explanation: Creates a Linear Regression model object.

Purpose: To prepare the regression algorithm for training.

Step 4: Train the Model

model.fit(X, y)

Explanation:

- The model learns patterns from the data.
- It calculates slope and intercept values.

Purpose: To train the algorithm so it can predict future energy output.

Step 5: Save the Model

pickle.dump(model, open("model.pkl", "wb"))

Explanation:

- Saves trained model to file.
- "wb" → Write in binary mode.

Purpose: To reuse the trained model in Flask without retraining.

Milestone 3: Model Testing

This milestone verifies whether the trained model works correctly before integrating into the web application.

File Used: predict.py

```
predict.py > ...
import joblib
import pandas as pd

# Load model
model = joblib.load("power_prediction.sav")

# Create input as DataFrame with column names
input_data = pd.DataFrame(
    [[900, 8.5]],
    columns=["TheoreticalPower", "WindSpeed"]
)

prediction = model.predict(input_data)

print("Predicted Power Output:", prediction[0])
```

Step 1: Load the Saved Model

```
model = joblib.load("power_prediction.sav")
```

Explanation: Loads trained model from saved file.

Purpose**:** To use the trained model for prediction testing.

Step 2: Create Input Data

```
input_data = pd.DataFrame(
    [[900, 8.5]],
    columns=["TheoreticalPower", "WindSpeed"]
)
```

Explanation: Creates structured input similar to training format.

Purpose: To test prediction using sample values.

Step 3: Generate Prediction

```
prediction = model.predict(input_data)
print("Predicted Power Output:", prediction[0])
```

Explanation: Model predicts energy output.

Purpose: To confirm model gives expected results.

Milestone 4: Flask Application Development

This milestone integrates the trained ML model into a web application using:

- Flask

Main file: app.py

The application allows users to interact through a browser.

```
pp.py > ...
from flask import Flask, render_template, request
import pickle
import numpy as np

app = Flask(__name__)

# Load trained model
model = pickle.load(open("model.pkl", "rb"))

# Dummy weather data (you can replace with API later)
city_weather = {
    "Hyderabad": {"temperature": 32, "humidity": 45, "pressure": 1012, "wind_speed": 5},
    "Chennai": {"temperature": 34, "humidity": 60, "pressure": 1009, "wind_speed": 6},
    "Mumbai": {"temperature": 30, "humidity": 70, "pressure": 1010, "wind_speed": 4},
    "Delhi": {"temperature": 28, "humidity": 50, "pressure": 1015, "wind_speed": 3},
    "Bangalore": {"temperature": 26, "humidity": 55, "pressure": 1013, "wind_speed": 5},
}

@app.route("/")
def home():
    return render_template("home.html")

@app.route("/dashboard", methods=["GET", "POST"])
def dashboard():
    weather = None
```

Step 1 : Import Required Libraries

from flask import Flask, render_template, request

import pickle

import numpy as np

Purpose: To enable web server functionality and prediction logic.

Step 2: Initialize Flask Application

app = Flask(__name__)

Explanation: Creates Flask app instance.

Purpose: To start web application server.

Step 3: Load Trained Model

model = pickle.load(open("model.pkl", "rb"))

Explanation: Loads saved ML model into memory.

Purpose: To use model for real-time predictions.

Step 4: Create Weather Dictionary

```
city_weather = {
    "Hyderabad": {"temperature": 32, "humidity": 45, "pressure": 1012, "wind_speed": 5},
}
```

Explanation: Stores predefined weather data.

Purpose: To simulate weather information without API.

Step 5: Create Home Route

```
@app.route("/")
def home():
    return render_template("home.html")
```

Explanation: Defines homepage URL.

Purpose: To display landing page.

Step 6: Create Dashboard Route

```
@app.route("/dashboard", methods=["GET", "POST"])
def dashboard():
```

Explanation: Handles form submissions and prediction logic.

Purpose: To manage user interaction and prediction**.**

Step 7: Handle Weather Button

```
if "get_weather" in request.form:
    city = request.form["city"]
    weather = city_weather.get(city)
```

Purpose: To display weather details of selected city.

Step 8: Handle Prediction Button

```
if "predict_energy" in request.form:
    theoretical_power = float(request.form["theoretical_power"])
```

```
wind_speed = float(request.form["wind_speed"])

input_data = np.array([[theoretical_power, wind_speed]])

prediction = round(model.predict(input_data)[0], 2)
```

Explanation:

- Takes user input
- Converts to numeric
- Sends to ML model
- Rounds result

Purpose: To generate predicted wind energy output.

Milestone 5: Frontend Development

User interface built using HTML.

Files:

- Home.html

```html
!DOCTYPE html>
html>
head>
   <title>Wind Energy System</title>
   <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}"
/head>
body class="home-body">

   <div class="hero-section">
       <h1>Next-Generation Wind Energy Prediction System</h1>
       <p>
           Intelligent wind forecasting and real-time energy prediction
           designed to power a smarter and sustainable future.
       </p>

       <a href="/dashboard" class="start-btn">Get Started</a>
   </div>

/body>
/html>
```

- Dashboard.html

```html
<!DOCTYPE html>
<html>
<head>
    <title>Dashboard</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
</head>
<body class="dashboard-body">

<h1 class="dashboard-title">Wind Energy Analytics Dashboard</h1>

<div class="container">

    <!-- LEFT SIDE -->
    <div class="card">

        <h2>📍 Location & Weather</h2>

        <form method="POST">
            <select name="city" required>
                <option value="">Select City</option>
                <option>Hyderabad</option>
                <option>Chennai</option>
                <option>Mumbai</option>
                <option>Delhi</option>
                <option>Bangalore</option>
            </select>

            <button type="submit" name="get_weather" class="btn-yellow">
                Get Weather
            </button>
        </form>
```

Home Page:

Purpose:

- Show project title
- Show description
- Provide navigation to dashboard

Dashboard Page:

Contains:

- Weather Section
- Prediction Section

Purpose:

To allow user to:

- View weather
- Predict wind energy

Milestone 5: Running the Application

Step 1: Open VS Code

Step 2: Run Command

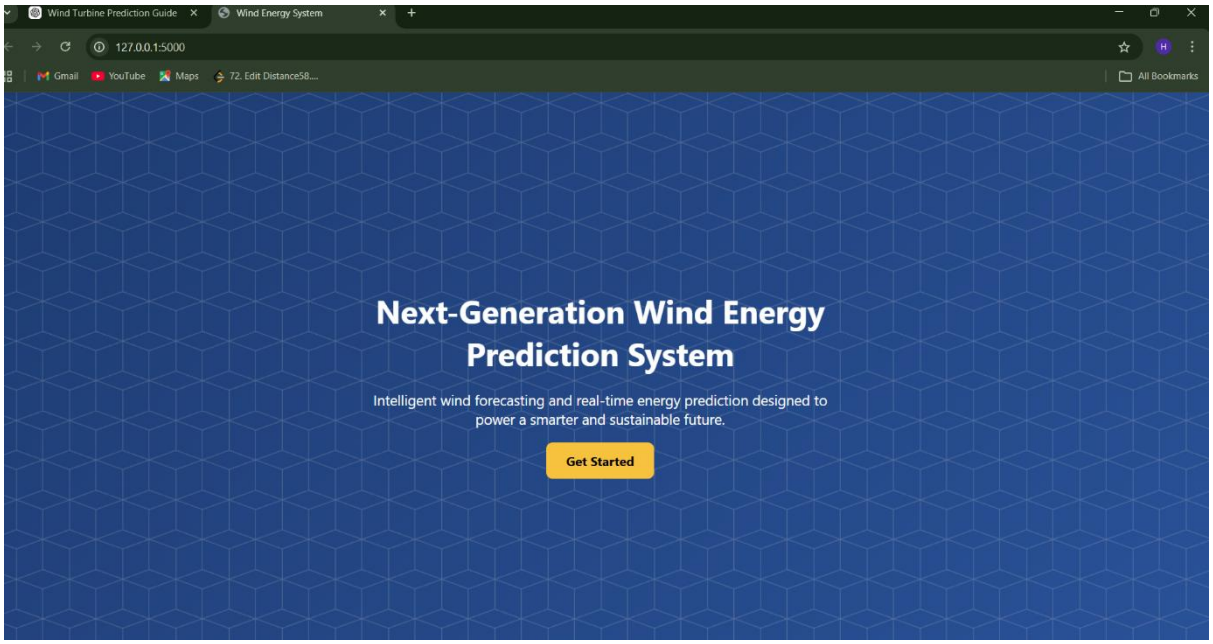
python app.py

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\DURGA PRASAD\Downloads\WindEnergyPrediction System-main> python train_model.py
Model trained and saved successfully!
PS C:\Users\DURGA PRASAD\Downloads\WindEnergyPrediction System-main> python app.py
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 139-412-803
127.0.0.1 - - [17/Feb/2026 16:12:37] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [17/Feb/2026 16:12:37] "GET /static/style.css HTTP/1.1" 304 -
127.0.0.1 - - [17/Feb/2026 16:12:38] "GET /favicon.ico HTTP/1.1" 404 -
```

Step 3: Open Browser

http://127.0.0.1:5000/ When the user clicks the Get Started link, the browser navigates to the /dashboard route. Flask handles this route and renders the dashboard.html template, displaying the dashboard page:



In your project, when the user clicks the “Get Started” button on the homepage (home.html), the application redirects (renders) the user to the Dashboard page (dashboard.html).

This navigation is handled by Flask routing.

This step connects the frontend interface with the backend route.

Step 1: **Button in home.html**

```
<a href="/dashboard" class="start-btn">Get Started</a>
```

Explanation:

- <a> → Anchor tag (used for navigation)
- href="/dashboard" → Redirects user to the /dashboard route
- When clicked, the browser sends a request to Flask server

Purpose: To navigate the user from the homepage to the dashboard page.

Step 2: Flask Route Handling in app.py

Your backend code:

```
@app.route("/dashboard", methods=["GET", "POST"])
def dashboard():
```

Explanation:

- @app.route("/dashboard") → Defines URL endpoint
- When browser requests /dashboard, this function executes
- methods=["GET", "POST"] allows:
    - GET → Normal page load
    - POST → Form submission

Purpose: To tell Flask what to do when `/dashboard` is requested.

Step 3: Rendering the Dashboard Page

Inside the dashboard function:

```
return render_template("dashboard.html",
                       weather=weather,
```

prediction=prediction)

Explanation:

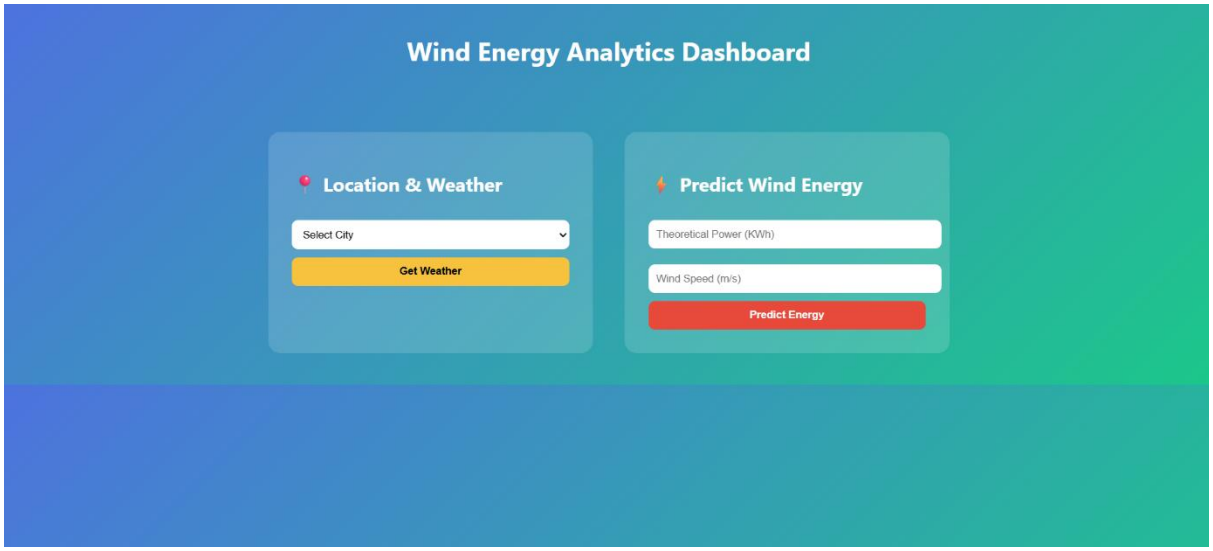
- render_template() loads dashboard.html
- Sends variables (weather, prediction) to HTML

Purpose: To display the dashboard page in the browser.

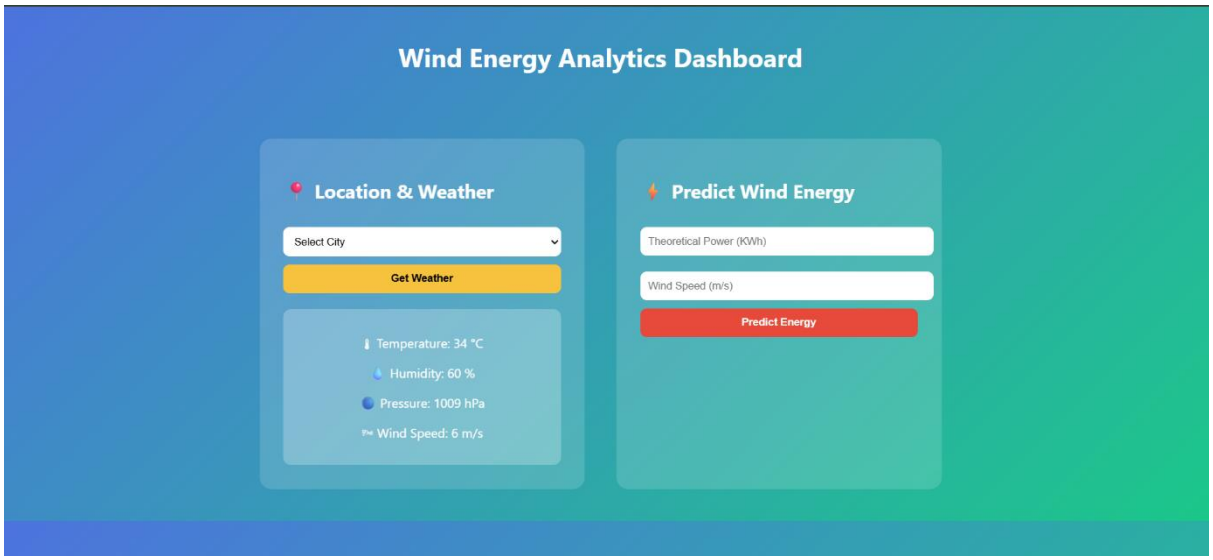
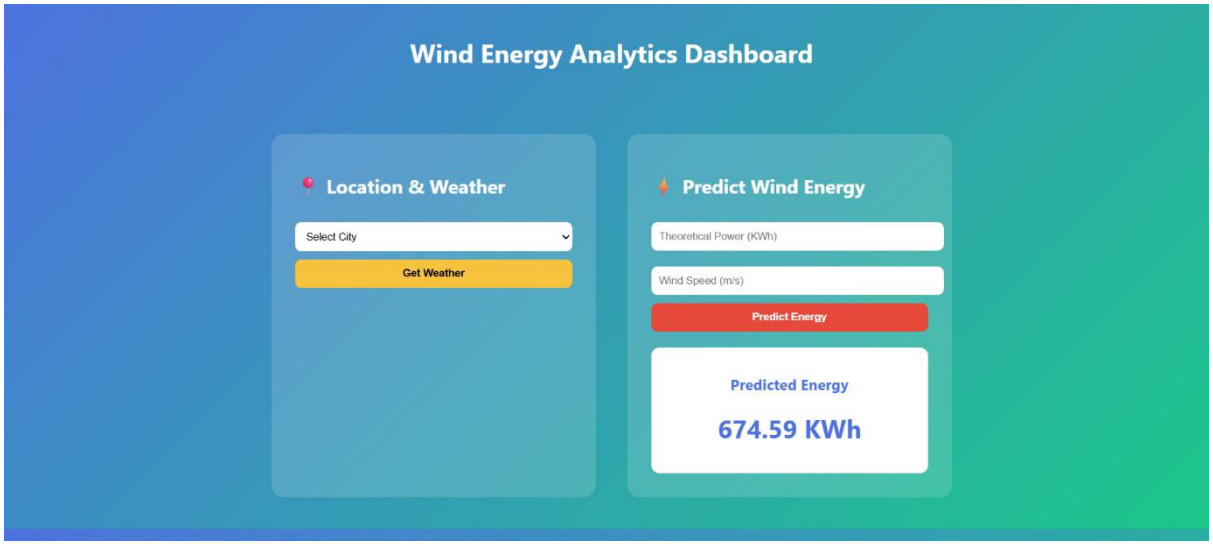
Complete Flow of "Get Started" Button

1. User opens homepage
2. User clicks **Get Started**
3. Browser sends GET request to /dashboard
4. Flask receives request
5. dashboard() function executes
6. dashboard.html is rendered
7. Dashboard page appears

When the user clicks the Get Started button, the anchor tag sends a request to the /dashboard route. Flask handles this request and renders the dashboard.html template. Therefore, the dashboard page loads successfully.



When the user selects a city and clicks Get Weather, the Flask backend retrieves predefined weather data from a dictionary and dynamically displays it on the dashboard page.



When the user enters theoretical power and wind speed and clicks Predict Energy, the backend receives the input, sends it to the trained Linear Regression model, and displays the predicted wind energy output on the dashboard.